DAY 1 · MODULE 1

Azure DevOps Foundations & End-to-End Traceability

Organising work in Azure Boards, and connecting every requirement to the change, the build and the release that delivered it.

Azure DevOps, CI/CD and AI-Assisted Release Readiness

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

ROADMAP

What We'll Cover

1. Why Delivery Needs Guardrails
2. The Azure DevOps Platform
3. Organising Work in Azure Boards
4. End-to-End Traceability
5. What's New: AI in Boards (optional)
6. Case Study
7. Knowledge Check
8. Recap

One thread runs through every section: a single change – story #42 – traced from the requirement to the release that delivered it.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

1

Why Delivery Needs Guardrails

Automation, traceability and quality as delivery non-negotiables

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

THE PREMISE

Speed Is Only Safe When the Process Makes It Safe

Automation
The same steps run the same way, every time

Traceability
Every change links to why, who, what and when

Quality checks
Tests, reviews and gates decide what ships

Result: each change is small, visible and verified – so a release becomes a non-event.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

THE PROBLEM

Four Things That Slow Delivery Down

1

Manual steps

Copying files, clicking through portals, editing configs by hand. Every manual step is a place for a mistake.

Fixed by automation

2

Missing links

Nobody can say which requirement a change belongs to, or what went into a release.

Fixed by traceability

3

Late testing

Defects found just before or after release are the slowest and costliest to fix.

Fixed by quality gates

4

Heroic releases

Weekend deployments that depend on one person who knows the steps.

Fixed by pipelines + approvals

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

DELIVERY NON-NEGOTIABLES

Three Pillars Hold Up Fast, Safe Delivery

Frequent, low-risk releases

Automation

Pipelines build, test and deploy every change the same way.

Modules 3 · 5 · 6

Traceability

Every change links to its requirement, review, build and release.

Modules 1 · 2

Quality

Tests, reviews and gates decide what ships – AI assists, humans approve.

Modules 4 · 6 · 7 · 8

Foundation: one integrated, auditable platform – Azure DevOps

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

THE DELIVERY LIFECYCLE

Every Stage of the Loop Has a Home in the Platform

Plan	Code	Build	Test	Release	Deploy	Operate	Monitor
01	02	03	04	05	06	07	08

Feedback from production shapes the next plan

Boards	Repos	Pipelines · Test Plans · Artifacts · Environments	Azure Monitor · App Insights

Each service can be used on its own – but used together they share identity, permissions and links, which is what makes end-to-end traceability possible.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

MEASURING DELIVERY

Five DORA Metrics Balance Throughput and Instability

THROUGHPUT – how quickly changes move

Change lead time
Commit → running in production

Deployment frequency
How often deployments happen

Failed deployment recovery time
Recovering from a failed deployment

INSTABILITY – how well deployments go

Change fail rate
Deployments needing a rollback or hotfix

Deployment rework rate
Unplanned deployments caused by incidents

The original 'four keys' are still common; DORA added rework rate and renamed time to restore.

Top performers are strong on both sides at once. Source: DORA – dora.dev/guides/dora-metrics.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

2

The Azure DevOps Platform

Five services, one project – and how organisations, access and security fit together

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

THE AZURE DEVOPS PLATFORM

One Project, Five Services That Share Everything

Shared across the project: identity · permissions · work items · links · history

1

Boards

Plan and track work: epics, stories, tasks, bugs, sprints

2

Repos

Git repositories, branches, pull requests and policies

3

Pipelines

CI/CD as YAML: build, test and deploy anywhere

4

Test Plans

Manual and exploratory testing linked to requirements

5

Artifacts

Package feeds: NuGet, npm, Maven, Python, Universal

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

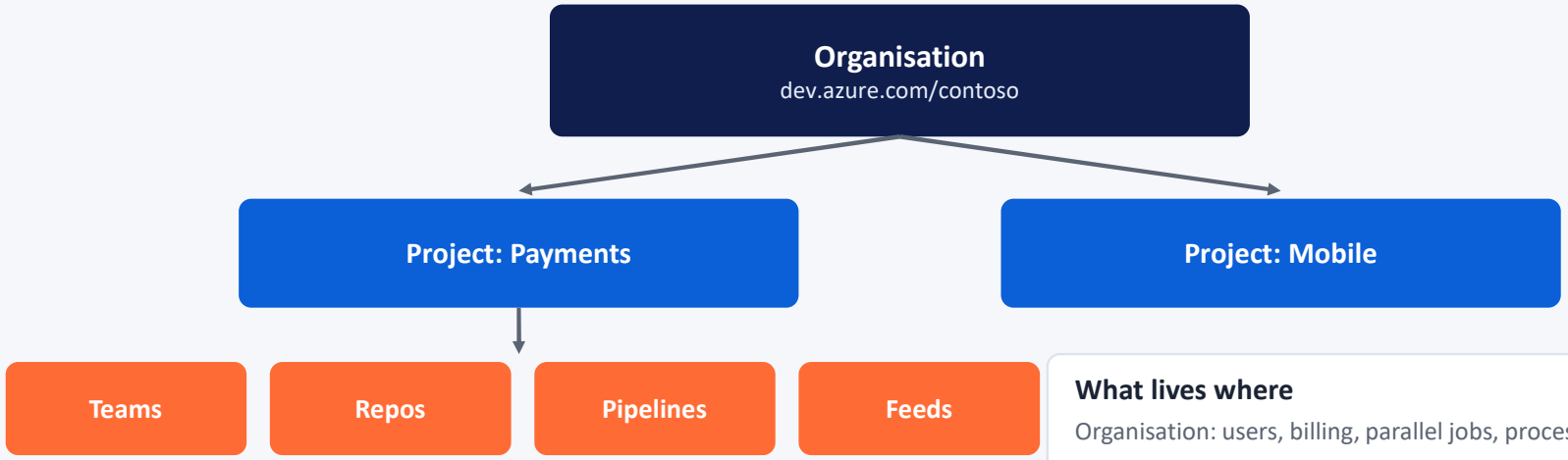
What Each Service Answers in the Delivery Flow

Service	Question it answers	Where we use it this week
Boards	What are we building, and why?	Module 1 – work items and traceability
Repos	What changed, who changed it, who reviewed it?	Module 2 – branches, PRs, policies
Pipelines	Does it build, pass the gates and deploy?	Modules 3, 4, 5, 6 and 8
Test Plans	Which requirements are tested, and how?	Module 4 – test traceability
Artifacts	Which exact package version was released?	Module 3 – versioned packages

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

HOW IT IS ORGANISED

Organisation, Projects and Teams



Lab: each learner owns an organisation named after their user, with one project – demo.

What lives where

Organisation: users, billing, parallel jobs, processes, policies

Project: work, code, pipelines, permissions

Team: backlog, board, area paths, sprints

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

ACCESS LEVELS

Access Level Decides Which Features a User Can Reach

Capability	Stakeholder	Basic	Basic + Test Plans
Work items and boards	Yes (limited)	Yes	Yes
Code in private projects	No	Yes	Yes
Pipelines and Artifacts	Limited	Yes	Yes
Run tests	No	Yes	Yes
Create and manage test plans	No	No	Yes
Cost	Free	First 5 users free	Paid (or VS Enterprise)

Access level controls features. Permissions (next) control what a user may do with them.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

SECURITY MODEL

Permissions Are Layered and Inherited Downwards

Organisation
Project Collection Administrators · Valid Users

Project
Project Administrators · Contributors · Readers · Build Administrators

Object
One repo, branch, pipeline, environment or service connection. Override inherited permissions only with a reason.

Good practice

1. **Grant to groups, not people**
2. **Least privilege by default**
3. **Use Microsoft Entra ID groups**
4. **An explicit Deny beats Allow**

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CONTEXT

Azure DevOps and GitHub Work Together

Azure DevOps
Boards · Repos · Pipelines · Test Plans · Artifacts

Boards ↔ GitHub repos
AB#123 links commits and PRs to work items

Pipelines ↔ GitHub repos
Build, PR validation and deploy from GitHub code

Copilot ↔ Boards and Repos
Start the coding agent from a work item; AI PR review

GitHub
Repos · Actions · Copilot · Advanced Security

In this programme code lives in Azure Repos – so commit mentions use #123. AB#123 is the syntax for code hosted in GitHub.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

3

Organising Work in Azure Boards

Work items, processes, states, teams and views

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

AZURE BOARDS

Work Breaks Down From Outcomes to Tasks

EPIC	Online ordering for business customers	*months · business outcome*
FEATURE	Checkout and order submission	*weeks · capability*
USER STORY	#42 Only active customers can submit an order	*days · user value*
TASK	Add status check · Write unit tests	*hours · how*
BUG	#57 Order total wrong at discount threshold	*a defect, planned like a story*

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

PROCESS TEMPLATES

Four Built-In Processes, One Choice per Project

Process	Requirement item	Other work item types	Fits teams that…
Basic	Issue	Epic, Task	want the simplest tracking
Agile ← this programme	User Story	Epic, Feature, Task, Bug, Issue	use stories and Kanban flow
Scrum	Product Backlog Item	Epic, Feature, Task, Bug, Impediment	run strict Scrum sprints
CMMI	Requirement	Epic, Feature, Task, Bug, Change Request, Issue, Review, Risk	need formal change control

Lab: projects start on Basic (Epic, Issue, Task). Case study step 1 switches the project to Agile.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

PROCESS CUSTOMISATION

Customise Through an Inherited Process, Never the Original

System process
Agile, Scrum, CMMI, Basic – locked

→

Inherited process
e.g. “CGI-Agile” – your changes live here

→

Projects
Every project on it updates immediately

Add
Custom fields, states, rules, layouts, work item types

Switch
Basic → Agile, Agile ↔ Scrum from Organization settings

Keep it light
Every required field is friction on every work item

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

WRITING GOOD WORK ITEMS

A Testable Story States Who, What, Why – and How We'll Know

USER STORY #42

As a sales operations lead

I want orders from inactive customers rejected

so that we do not ship to closed accounts.

Story points 3 · Priority 2 · Area: Orders

```
# acceptance criteria
Given a customer with status Inactive
When  they submit an order
Then  the order is rejected
And   no order record is created

# each line can become a test
```

Independent | **Valuable** | **Small** | **Testable**

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

TWO DIFFERENT CHECKLISTS

Acceptance Criteria vs Definition of Done

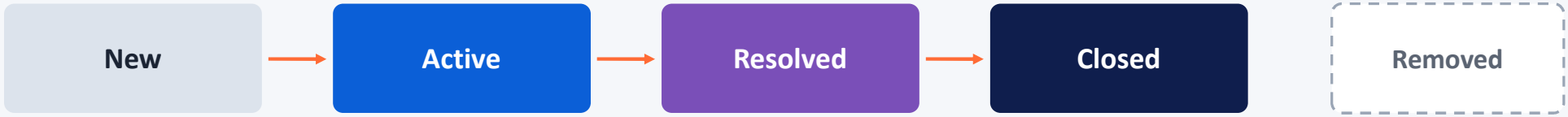
Acceptance criteria – unique to story #42	Definition of Done – same for every story	
Inactive customer’s order is rejected	Reviewed in a pull request	Unit tests pass in the pipeline
No order record is created	Coverage gate met	Linked to its work item
Active customer’s order still succeeds	Deployed to the test environment	No critical security findings

By Friday, most of the Definition of Done is enforced by the platform – policies, gates and checks – not by memory.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

WORKFLOW

Every Work Item Moves Through a Defined Set of States



State category	Agile story state	What it means for reporting
Proposed	New	On the backlog, not started
In Progress	Active	Being worked on – counts toward work in progress
Resolved	Resolved	Code done, awaiting verification
Completed	Closed	Done – counts toward velocity and throughput

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CLASSIFYING WORK

Area Paths Say What and Who; Iteration Paths Say When

```
# Area path – what / who
demo
├─ demo\Orders
│   ├─ demo\Orders\Checkout
│   └─ demo\Orders\Billing
└─ demo\Mobile
```

Sprint 1
Done

Sprint 2
Current

Sprint 3
Next

```
# Iteration path – when
demo\Release 1\Sprint 2
```

Area path

Product area or component. Decides which team's backlog an item lands on.

Iteration path

A time box with dates. Drives sprint backlogs, capacity and burndown.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

TEAM CONFIGURATION

Each Team Tunes How Boards Works for It

1

Areas
Which area paths feed the team's backlog

2

Iterations
Which sprints the team plans in

3

Backlog levels
Epics / features / stories shown; how bugs appear

4

Capacity
Hours per person per day, days off, activity

5

Board columns
Custom columns, WIP limits, Definition of Done

6

Working days
Used by burndown and capacity charts

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

WORKING VIEWS

Backlog to Plan, Board to Flow, Taskboard to Deliver

Backlog

1 #42 Active customer check

2 #44 Order history page

3 #57 Discount bug

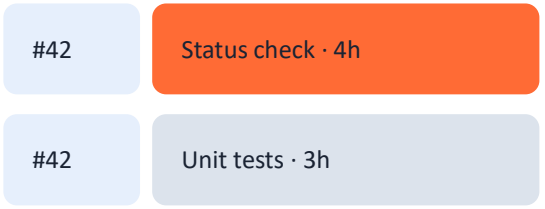
Ordered list. Drag to prioritise; drag into a sprint to plan.

Kanban board



Visualise flow, limit work in progress, spot blockers.

Sprint taskboard



Tasks per story with remaining hours for this sprint.

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

QUERIES AND DASHBOARDS

Queries Find Work; Dashboards Keep It Visible

```
# Open stories this sprint with no linked code
SELECT [System.Id], [System.Title], [System.State]
FROM WorkItems
WHERE [System.WorkItemType] = 'User Story'
  AND [System.State] <> 'Closed'
  AND [System.IterationPath] = @CurrentIteration
  AND [System.ExternalLinkCount] = 0
```

Flat list
Items matching a filter

Tree of work items
Parent/child, e.g. features with stories

Work items and direct links
Items and what they link to – best for traceability

Save → chart → pin to a dashboard

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

4 End-to-End Traceability

From requirement to commit, pull request, build and release

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

TRACEABILITY

Traceability Answers One Question: Why Is This Code in Production?

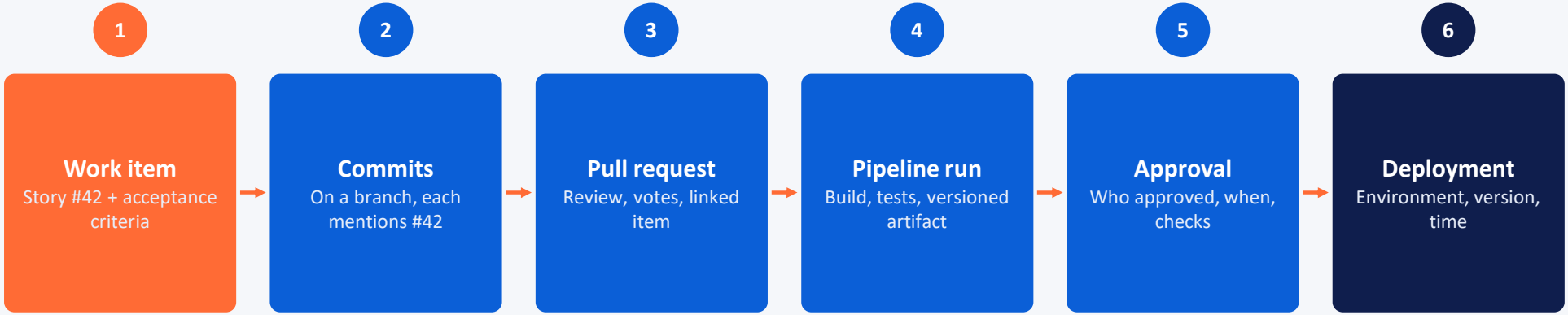
Who asked for it?	Who reviewed it?	Was it tested?	Who approved release?
↓	↓	↓	↓
Work item	Pull request	Pipeline run	Environment approval

Goal: answer all four in minutes – during an incident or an audit.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

THE TRACEABILITY CHAIN

Six Links From Requirement to Production



In Azure DevOps these links are created as a side effect of normal work – branch, commit, pull request, merge, deploy. No spreadsheet, no extra step.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CREATING THE LINKS

How Each Link Gets Created – and Where You See It

Link	How it is created	Where you see it
Branch	'Create a branch' from the work item	Work item → Development
Commit	#42 in the commit message (Commit mention linking on)	Work item → Development
Pull request	Link in the PR, or #42 in its description	PR → Work items panel
Build	Automatic from the run's commits (+ 'Automatically link new work in this build')	Run → Related; work item → Development
Deployment (YAML)	Automatic, when a run deploys to an environment	Environment → deployment history
GitHub repo	AB#42 in commit or PR text	Work item → Development

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

COMMAND PROMPT / TERMINAL

A Developer's Normal Workflow Builds the Trail

```
# start from the work item's branch
git switch -c feature/42-active-customer

# commit, mentioning the story
git commit -m "Reject orders from inactive customers #42"
git commit -m "Add tests for status check #42"

# publish, then open a pull request
git push -u origin feature/42-active-customer
```

STORY #42 · DEVELOPMENT

1. **Branch** feature/42-active-customer
2. **Commits** 2 commits mentioning #42
3. **Pull request** !17 – completed
4. **Build** 20261028.3 – succeeded
5. **Deployed** see environment history (Module 5)

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

LINK TYPES

Work Links Connect Items; Development Links Connect Code

Between work items

Parent / Child
roll-up hierarchy

Related
loose association

Predecessor / Successor
dependencies

Tests / Tested By
test coverage

Duplicate / Duplicate Of
clean-up

To development objects

Branch
where work happens

Commit
the change itself

Pull Request
the review

Build / Integrated in build
the run that contained it

Found in build
for bugs

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

BUILT-IN TRACEABILITY VIEWS

Five Places to Read the Trail Without Leaving Azure DevOps

1

Work item form
Development panel: branch, commits, PR, build

2

Pull request
Linked items, reviewers, votes, policies

3

Pipeline run
Changes, related items, tests, artifacts

4

Environment
YAML deployment history: run, commits, work items

5

Queries
'Work items and direct links' reports

Start anywhere
and walk to all the others

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

TWO DIRECTIONS

Trace Forward to Prove Delivery, Backward to Explain Incidents

Forward – "Is story #42 live yet?"



Backward – "What caused last night's incident?"

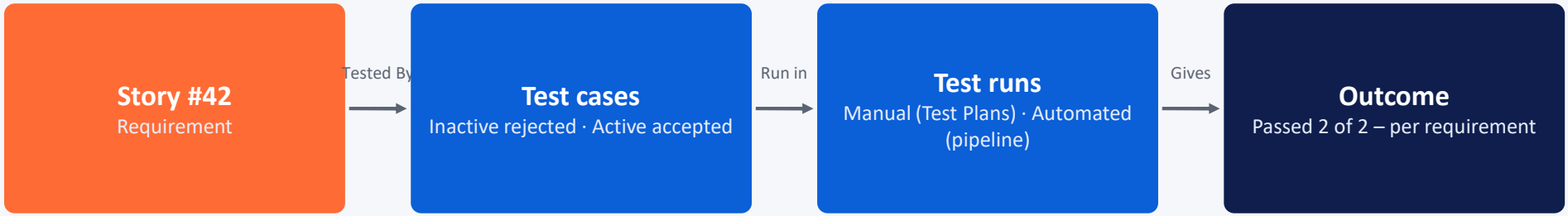


Backward tracing is where weak traceability hurts most – during an incident nobody has time to search chat history.

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

REQUIREMENTS AND TESTS

Linking Tests to Requirements Shows What Is Actually Verified



Requirement-based suites
pull in every test linked to a story

Automated tests
can be associated with test cases and run from a pipeline

Gaps show up
stories with no linked tests are easy to query

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

SYMPTOM → CAUSE → FIX

Common Ways Traceability Breaks

What breaks it	Why it hurts	Better practice
Commits with no work item ID	Chain breaks at the first link	#ID habit + linked-work-item PR policy
One giant PR for many stories	Can't map change to requirement	Small PRs, one story each
Hotfixes pushed straight to main	No review, no trail, highest risk	Bug work item + short PR, even in an emergency
A spreadsheet tracking releases	Drifts from reality within days	Environment deployment history
Everything linked as 'Related'	No hierarchy, no roll-up	Parent/Child for hierarchy
Items closed long after release	Reports are wrong	Resolved at PR merge; Closed when the DoD is met

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

TRACEABILITY AS EVIDENCE

Every Audit Question Already Has an Answer in the Platform

Who requested the change?	Work item – creator, history, criteria
What exactly changed?	Commits – diff, author, time
Who reviewed and approved the code?	Pull request – reviewers, votes, policies
Was it tested before release?	Pipeline run – tests, quality gates
Who approved production?	Environment approval – approver, time
When did it reach production?	Deployment history – environment, version

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

5

What's New: AI in Boards

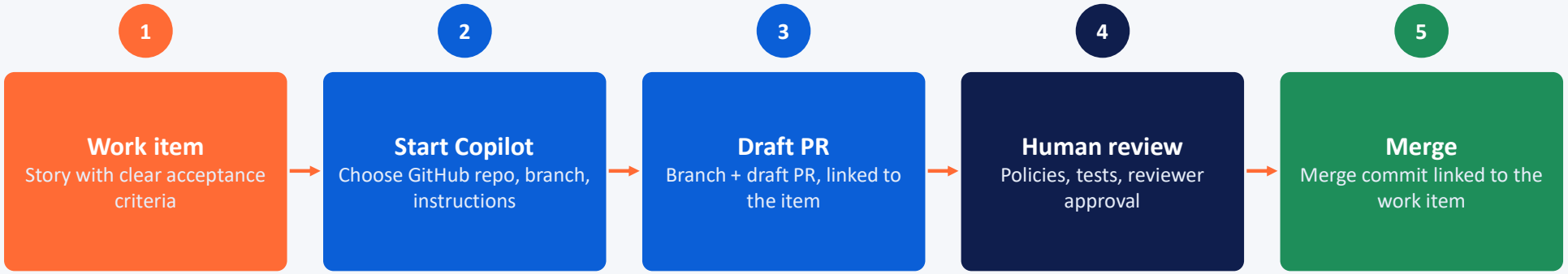
Optional awareness – Copilot from a work item, recent Boards changes and the Azure DevOps MCP Server

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

OPTIONAL · PICTORIAL — AI IN BOARDS

Starting GitHub Copilot From a Work Item



Requires: code in a GitHub repository · Boards connected via the GitHub App · a GitHub Copilot subscription. Not available for Azure Repos.

AI-generated work stays inside the same traceability chain – and its quality depends on how well the acceptance criteria are written.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

OPTIONAL · RECENT CHANGES

What Changed in Azure Boards in 2026

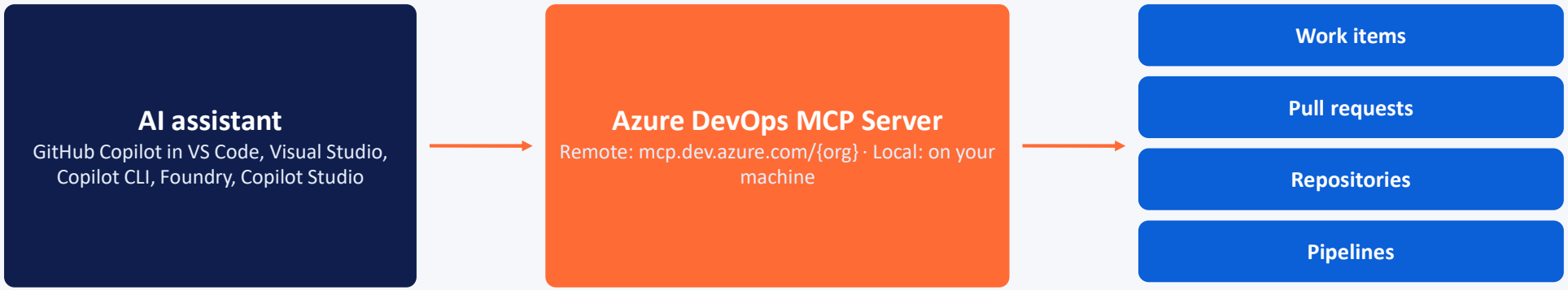
Change	What it does	When
Copilot integration for Boards (GA)	Start the coding agent from a work item; draft PR linked back	Sprint 268 · Jan 2026
Markdown editor for work items	Markdown fields open in preview; edit icon to change	Sprint 272 · Apr 2026
Filter backlogs and boards by more fields	Any field on columns or cards can be a filter	Sprint 272 · Apr 2026
Sprint board and backlog filters	Filter sprints by additional and custom fields	Sprint 274 · May 2026
PR annotations on board cards	Linked PR status visible on cards, on by default	Sprint 277 · Jul 2026
Flexible work item copying	Copy parent links, child links or both	2026

Source: Azure DevOps release notes (learn.microsoft.com/azure/devops/release-notes). Features roll out over 2–3 weeks.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

OPTIONAL · PICTORIAL — AI ASSISTANTS

The Azure DevOps MCP Server Lets AI Assistants Work With Boards



Remote server
Generally available 5 Aug 2026

Local server
Generally available Oct 2025

Identity
Microsoft Entra ID – your own permissions

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

6

Case Study

Set up a project and trace one change end to end

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CASE STUDY

Set Up a Project and Trace One Change End to End

1. Switch the project process from Basic to Agile
2. Create an epic, a feature and a user story with acceptance criteria
3. Create a branch from the story; commit with #ID
4. Open a pull request linked to the story and complete it
5. Run a starter YAML pipeline; find the story under Related
6. Trace back from the run to the story – and forward again

Work in your own organisation and the project "demo". About 30 minutes.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CASE STUDY

What the Finished Trail Looks Like

Boards	Repos	Pipelines
Epic → Feature → Story with acceptance criteria, now Resolved. Development panel: branch, commits, PR, build.	Branch named after the story; commits ending in #ID; completed PR with the story in its Work items panel.	Run numbered by date (e.g. 20261028.1); summary shows ‘1 work item’ under Related.

Done when: from the pipeline run you reach the story’s acceptance criteria in three clicks – and walk back again.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

TRAINER DEMO · 5 MINUTES

Following a Deployment Back to the Story

Environment
staging-demo – deployment history

→

Deployment
When, which run, which stage

→

Pipeline run
Commits and related work items

→

Story
Acceptance criteria – the 'why'

YAML pipelines (this programme)

Deployments are recorded on the environment with their runs, commits and work items. No Azure resources needed for the demo – a deployment job is enough.

The work item's Deployment control

Filled by Classic release pipelines ('Report release status to Boards') – not by YAML environments.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

SYMPTOM → CAUSE → FIX

Lab Issues You Are Most Likely to Hit

Symptom	Cause	Fix
No User Story in New Work Item	Project still on the Basic process	Change process to Agile
Commit not linked to the story	AB# used, or Commit mention linking off	Use #ID; check repo settings
TF402455 when saving YAML	Branch policy blocks direct pushes to main	Commit to a new branch and use a PR
Run stays 'Queued'	Job busy, permission pending, or no hosted-job grant	Read the run message; Parallel jobs page; ask IT
Warning about ubuntu-latest	Label moving to Ubuntu 26 (from 19 Oct 2026)	Pin vmImage: ubuntu-24.04
'demo is empty' in Repos	Repository has no commits yet	Initialize with a README

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

7

Knowledge Check

Seven scenario questions – think before you answer

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

KNOWLEDGE CHECK · QUESTION 1 OF 7

Question 1

A team's project uses the Basic process. They want Features and User Stories without starting a new project or losing existing work items. What is the most direct supported way?

A. Edit the Basic system process and add the Feature and User Story types

B. Create a new Agile project and move the work items across

C. Change the project's process to Agile from Organization settings › Process

D. Rename the Issue type to "User Story" in project settings

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 1

Answer 1: C

A **Not this – Edit the Basic system process and add the Feature and User Story types**
System processes are locked – they cannot be edited at all.

B **Not this – Create a new Agile project and move the work items across**
Works, but it is not the most direct path and loses project history and settings.

C **Correct – Change the project's process to Agile from Organization settings › Process**
Basic → Agile is a supported process change, and existing items keep their history.

D **Not this – Rename the Issue type to "User Story" in project settings**
Work item types can't be renamed this way, and it wouldn't add the Feature level.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · QUESTION 2 OF 7

Question 2

A developer pushes a commit to Azure Repos with the message “Fix null check #123”. Ten minutes later, story 123 still shows nothing in its Development section. What should you check first?

A. Whether the developer’s personal access token has the Work Items scope

B. Whether story 123 is assigned to the current sprint

C. Whether a pipeline has run for that commit yet

D. Whether the repository setting “Commit mention linking” is turned on

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 2

Answer 2: D

A **Not this – Whether the developer's personal access token has the Work Items scope**
Linking from a commit message isn't done with the developer's token – the service creates it.

B **Not this – Whether story 123 is assigned to the current sprint**
Sprint assignment has no effect on development links.

C **Not this – Whether a pipeline has run for that commit yet**
The commit link doesn't need a build; the build link is a separate, later link.

D **Correct – Whether the repository setting "Commit mention linking" is turned on**
#ID mentions only create links when this repository setting is on – the first thing to check.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · QUESTION 3 OF 7

Question 3

After a reorganisation, Team A's backlog suddenly shows work that belongs to Team B. Sprint dates and sprint assignments look correct. What most likely changed?

A The area paths owned by Team A, or the area path on those items

B The iteration paths selected in Team A's settings

C The project's process template

D The Kanban column settings on Team A's board

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 3

Answer 3: A

A **Correct – The area paths owned by Team A, or the area path on those items**
Area paths decide which team's backlog an item lands on – a reorg typically changes them.

B **Not this – The iteration paths selected in Team A's settings**
Iterations decide when work is planned, not which team's backlog it appears on – and sprints look right.

C **Not this – The project's process template**
A process change alters types and states, not which team sees which items.

D **Not this – The Kanban column settings on Team A's board**
Board columns change how items are displayed, not which items are included.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

Question 4

An error spike starts at 02:10. All you know is that something was deployed to production a few minutes earlier. Which starting point gets you to both the approver and the original requirement in the fewest steps?

A. Search the repository's commit history for commits made the previous evening

B. Open the production environment's deployment history, then the run, its approval and related work items

C. Run a Boards query for stories closed in the last week

D. Filter completed pull requests by date and read each description

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 4

Answer 4: B

A **Not this – Search the repository's commit history for commits made the previous evening**
Commits don't tell you what was deployed when, or who approved it.

B **Correct – Open the production environment's deployment history, then the run, its approval and related work items**
The deployment record ties time → run → approval → linked work items: backward tracing.

C **Not this – Run a Boards query for stories closed in the last week**
Closed stories may not be deployed yet, and this doesn't give you the approver.

D **Not this – Filter completed pull requests by date and read each description**
PRs show reviews, not which deployment went out or who approved production.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

Question 5

To 'reduce risk', a team adds a weekly change-board meeting that must approve every production deployment. Which two DORA metrics will this most directly make worse?

A Change fail rate and failed deployment recovery time

B Change lead time and change fail rate

C Deployment frequency and deployment rework rate

D Deployment frequency and change lead time

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 5

Answer 5: D

A **Not this – Change fail rate and failed deployment recovery time**
Both describe how well deployments go or how fast you recover; a weekly meeting doesn't directly change them (recovery could suffer if fixes must wait – a fair nuance).

B **Not this – Change lead time and change fail rate**
Lead time worsens, but there's no direct reason for the fail rate to change – bigger batches may even make it worse indirectly.

C **Not this – Deployment frequency and deployment rework rate**
Frequency drops, but rework rate counts unplanned incident-driven deployments – not the direct effect.

D **Correct – Deployment frequency and change lead time**
Batching into a weekly slot cuts deployment frequency and makes every change wait longer to reach production.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

Question 6

A QA lead must create test plans and requirement-based suites linked to stories, but never pushes code. What is the minimum they need?

A. Stakeholder access level

B. Basic access level

C. Basic + Test Plans access level

D. Project Administrator permissions with Basic access

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 6

Answer 6: C

A **Not this – Stakeholder access level**
Stakeholders can work with work items but can't author test plans.

B **Not this – Basic access level**
Basic users can run tests but not create or manage test plans and suites.

C **Correct – Basic + Test Plans access level**
Authoring test plans and suites needs the Basic + Test Plans access level (or a trial / VS Enterprise).

D **Not this – Project Administrator permissions with Basic access**
Permissions don't unlock licensed features – this mixes up permissions and access levels.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · QUESTION 7 OF 7

Question 7

Your team keeps code in Azure Repos and wants to start GitHub Copilot's coding agent directly from Azure Boards work items. What happens?

A. It isn't supported – the integration needs the code in a GitHub repository, connected via the GitHub App

B. It works once a personal access token with Code scope is added

C. It works, but only on Bug work items

D. It works if the team has the Basic + Test Plans access level

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 7

Answer 7: A

A **Correct – It isn't supported – the integration needs the code in a GitHub repository, connected via the GitHub App**
The Boards–Copilot integration creates branches and draft PRs in GitHub repositories only.

B **Not this – It works once a personal access token with Code scope is added**
PATs aren't supported for this integration – the GitHub App connection is required.

C **Not this – It works, but only on Bug work items**
It supports types in the Requirements and Task categories, not only bugs – but repo type is the blocker.

D **Not this – It works if the team has the Basic + Test Plans access level**
Access level isn't the issue; the repository type is.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

WRAP-UP

Recap

Topic	Key takeaway
Why guardrails	Speed is safe only with automation, traceability and quality – DORA's five metrics cover throughput and instability
Platform	Five services in one project share identity, permissions and links; access level = features, permissions = rights
Azure Boards	Epic → Feature → Story → Task; one process per project (labs use Agile); area = who/what, iteration = when
Good work items	Testable acceptance criteria per story; a shared Definition of Done enforced by the platform
Traceability	Six links, created by normal work: #ID commits, linked PRs, build links, environment deployment history
Reading the trail	Forward to prove delivery, backward to explain incidents; audit questions map to platform records
What's new	Copilot from work items (GitHub repos), better filters, PR status on cards, MCP server – AI stays in the chain

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.